Presentable Debugging with LLVManim

Benjamin Houle
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Benjamin_Houle@student.uml.edu

Ebenezer Oloyede
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Ebenezer_Oloyede@student.uml.edu

Lily Grippo
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
Lily_Grippo@student.uml.edu

Mohammad Ahmed
Dept. of Computer Science
University of Massachusetts Lowell
Lowell, Massachusetts, USA
ahmedzubair16dec@gmail.com

***Abstract*—LLVManim is a code visualization tool that generates aesthetically pleasing animations for information control flow and memory allocations. Utilizing the well-supported Manim library alongside LLVM's built-in static analysis tools and wide language support, LLVManim is able to quickly create informative, professionally styled animations for any code that compiles to LLVM IR. The primary objective of the project is to create an alternative to the unappealing "programmer art" and visual clutter that many standard tools suffer from, instead focusing on an output more suitable for presentations or educational purposes.**

***CCS Concepts:* • Human-centered computing → Visualization → Visualization systems and tools → Visualization toolkits; • Software and its engineering → Software notations and tools → Compilers; • Applied computing → Education → Interactive learning environments**

***Index Terms*—program visualization, code animation, LLVM, static analysis, control flow graphs, memory visualization, educational tools, Manim, compiler infrastructure, presentation tools**

I. Introduction

Despite static analysis and data visualization being among the oldest concepts in computer science, it seems that disappointingly few graphic designers have been significantly interested in the former. While plenty of tools exist for creating control flow graphs or visualizing the stack, the majority of these favor a "function over form" paradigm that focuses on creating light-weight readable output, assuming the user is more concerned with the information than its presentation. This, however, is not always the case: presenting in any context, such as at a conference or school, generally favors quickly digestible and visually appealing content opposed to static, large blocks of text. In these cases, the presenter is currently forced to choose between using the unappealing generated visualization and the slog of creating something presentable for themselves.

II. Existing Tools

A. PythonTutor

PythonTutor [1] is a widely used educational tool for visualizing program execution, originally developed to help novices build accurate mental models of how code runs. Its core contribution is a step-by-step execution trace that exposes stack frames, heap objects, and control flow in a concrete, visual manner. As described by its creator, Philip J. Guo, PythonTutor aims to "help people overcome a fundamental barrier to learning programming: understanding what happens as the computer runs each line of code" [2].

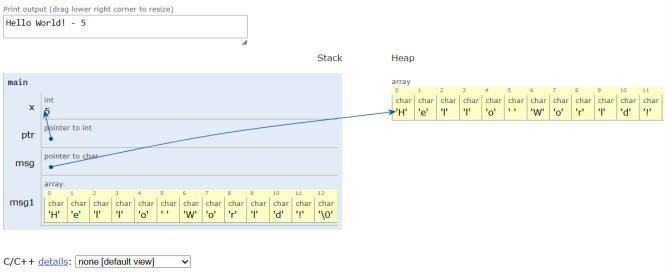

Fig. 1. Example Pythontutor.com output.

In fact, PythonTutor has been shown to be effective in improving students' understanding of programming concepts; Guo's further studies demonstrated significant learning gains when PythonTutor was used as a supplement to traditional instruction [3]. This shows the educational value of connecting code execution to visual representations, as well as the benefits of an accessible, interactive tool.

However, the visualizations produced by PythonTutor are relatively basic and not suitable for presentation purposes. PythonTutor's primary focus is on clarity and educational value rather than aesthetics, resulting in output that may be difficult to read or visually unappealing when projected to an audience. This, along with the fact that the tool is web-based and not designed for offline use, makes it less than ideal for use in settings such as conference presentations where visual impact is important. PythonTutor is additionally restricted by its limited customization options, minimal library

of visualization methods, and support for only C, C ++, Java, Javascript, and Python.

B. LLVM Analysis and Transform Passes

LLVM has emerged as the de-facto standard compiler infrastructure for modern software development, powering everything from the development environment for Apple operating systems to cutting-edge systems research efforts. Its adoption by the compiler community can be attributed to several key strengths–a modular architecture that separates language frontends from optimization backends, an extensive suite of analysis and transformation passes that expose deep insights into program behavior, and robust support for numerous programming languages including C, C++, Rust, Swift, and Julia [4].

At the core of LLVM's value proposition for code analysis is its Intermediate Representation (IR), a low-level and platform-independent format that captures program semantics in a standardized way. This IR serves as the foundation for powerful built-in analysis passes that can generate control flow graphs, dominator trees, data flow analyses, and detailed memory access patterns. For compiler developers and researchers, these capabilities are invaluable—LLVM effectively solves the hard problem of extracting structured information from code, allowing them to focus on higher-level tasks like optimization or verification [5].

However, LLVM's visualization capabilities tell a different story. The standard approach for visualizing analysis results relies on generating DOT[1] files through passes like `-dot-cfg` or `-view-cfg`, which are then rendered using Graphviz [6]. While functionally correct, the resulting diagrams embody a strictly utilitarian aesthetic: monochrome nodes and edges, densely packed labels filled with LLVM IR instructions, and layouts optimized for completeness rather than clarity. This "function over form" philosophy serves well enough as a developer tool for debugging optimizer passes, but it fails catastrophically as a solution for communication beyond that narrow domain. A lecturer explaining recursion to undergraduates, a tech lead presenting architectural decisions to stakeholders, or a researcher delivering a conference talk all face the same problem: LLVM-generated visualizations are too dense, too technical, and frankly, too ugly to serve their pedagogical or presentational needs.

The barrier extends beyond mere aesthetics. Interpreting these diagrams requires familiarity with LLVM IR syntax—understanding what "`%4 = load i32, i32* %3`" means demands compiler knowledge that most audiences lack. The visualizations are fundamentally static images with no mechanism for progressive disclosure or animation, forcing presenters to mentally simulate execution flow while pointing at a frozen graph. Moreover, customization options are severely limited; changing colors, adjusting layouts, or highlighting specific execution paths requires manual post-processing in external tools, completely defeating the purpose of automated generation.

[1]Abstract grammar for defining Graphviz nodes, edges, graphs, subgraphs, and clusters.

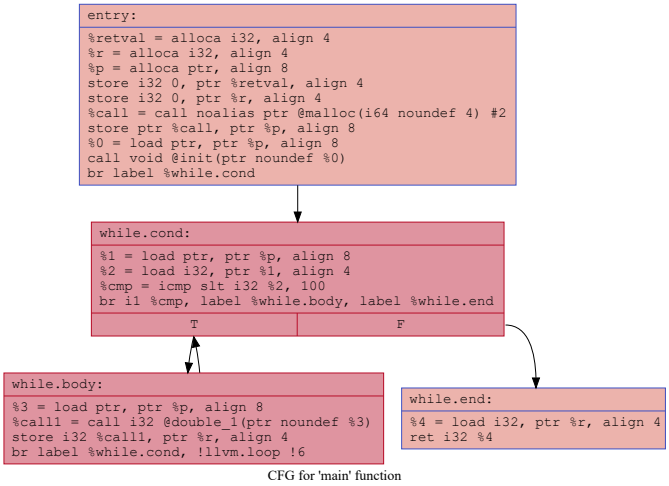

Fig. 2. Example Control Flow Graph using LLVM `-dot-cfg` pass. While technically accurate, the visualization prioritizes information density over visual clarity.

In essence, LLVM provides extraordinarily powerful analysis capabilities but pairs them with visualization tools that assume their primary consumer is someone who already understands what they're looking at. This creates a frustrating gap: we can automatically extract rich semantic information about program behavior, but we cannot automatically present that information in a way that serves educational or presentational contexts. LLVManim aims to bridge this gap by maintaining LLVM's analytical rigor while replacing its austere output with animations designed for human communication rather than machine precision.

III. Manim

The Manim library [7] was originally developed by Grant Sanderson [8] for his popular educational YouTube channel "3Blue1Brown" to create animations of mathematical and computer science concepts. [9] His approachable animations have helped him become one of the most well-known online educators in these fields and have helped countless others understand complex topics more easily. Given this history, Manim represents an excellent candidate for creating high-quality educational presentations. Unfortunately, many potential users lack either the ability or the willingness to invest the substantial time required to create custom animations with this Python library.

While tools like PythonTutor and LLVM successfully address the technical challenge of extracting and displaying program information, they fall short in making that information genuinely engaging. This is where Manim distinguishes itself, not as a code analysis tool, but as a demonstration of what becomes possible when visualization prioritizes both clarity and aesthetic appeal.

Since its inception in 2015, the 3Blue1Brown channel has gained more than 6 million subscribers and hundreds of

millions of views. Individual videos on topics such as linear algebra, calculus, and even neural networks regularly reach audiences of millions. Sanderson's approach, using smooth, precisely choreographed animations to build intuition about abstract concepts, has earned him widespread recognition in the educational community. The key to all of these animations is Manim, an animation engine written in Python and maintained by Sanderson himself.

Manim's strength lies in its ability to programmatically generate animations with mathematical precision. Instead of tiresome keyframe animation like other software, Manim allows users to write Python code to determine how different mathematical objects should look, change, and interact with each other. Manim can render beautiful 3D scenes, render LaTeX equations, and output crisp 60 fps animations for any purpose, from YouTube tutorials to academic conferences. What makes Manim unique for education is its capacity to make complicated ideas, like matrix transformations, function composition, or even program flow, tangible and understandable to the viewer by systematically unfolding them.

Yet, this power comes with a barrier to entry. It is not trivial for a professor who wants to make a presentation for a single lecture, or a researcher who wants to create visualizations for a conference presentation, to learn and use Manim effectively. They may have a clear idea of what they want to show, perhaps how a recursive function builds up its stack, but writing Manim code for it requires a distinct skill set. LLVManim addresses this challenge by automatically translating program semantics into visual representations, thereby eliminating the barrier of manual coding between LLVM's analytical power and Manim's visual sophistication.

IV. LLVMANIM

By providing a solution to quickly and easily create data-based animations, LLVManim aims to bridge the gap between these two concepts:

1) Methods exist to quickly generate code visualizations, but they are unappealing for presentation purposes.
2) The Manim library can make visually impressive animations, but using it is time consuming.

Educators and tech leads alike can rejoice in spicing up their PowerPoints without needing to compromise between style and accessibility.

A. Project Goals

The primary goal of LLVManim is to create a lightweight pipeline for converting LLVM IR and runtime traces into concise, animated explanations of program behavior. As a proof-of-concept, the initial version of LLVManim will focus on visualizing fundamentals, such as control flow and memory allocations.

Proof-of-Concept Targets:

- **IR Instruction Coverage:** Support for the most common LLVM IR instructions, such as `load`, `store`, `call`, and basic arithmetic operations. This will allow LLVManim to visualize a wide range of simple programs.
- **Trace Event Mapping:** Correctly map $\geq 95\%$ of runtime events (loads, stores, branches, calls, returns) to IR locations when debug metadata is present.
- **Customization Options:** Provide ≥ 4 user-configurable options – Analysis method, animation speed, color palette, & verbosity.
- **Usability:** In a pilot study (6–8 participants), achieve $\geq 80\%$ task completion for locating injected bugs.

Core Features:

- **LLVM IR Parsing:** Extract control flow and memory access information from LLVM IR, utilizing debug metadata when available.
- **Manim Animation Generation:** Translate LLVM IR and trace information into Manim scenes that visually represent control flow, stack frames, heap allocations, and data flow.
- **User Interface:** Develop a simple command-line interface that allows users to specify input files, select visualization options, and generate animations with minimal configuration.
- **Basic Animation Customization:** Allow users to customize the appearance of animations (e.g. color schemes, animation speed, etc.) to suit different presentation contexts.

Major features (required to realize the intended product):

- **Advanced Animation Customization:** Allow users to customize more aspects of the animations, such as layout styles, node shapes, or animation effects.
- **Basic Interactive Features:** Implement basic interactivity in the generated animations, such as pausing, stepping through execution, or highlighting specific paths.

Stretch features (nice-to-have if time permits):

- **GUI:** Create a graphical user interface for users who prefer not to use the command line.
- **Advanced Interactive Features:** Implement advanced interactivity in the generated animations, such as allowing users to click on nodes to see underlying IR code or variable values.

B. Implementation Plan

We approach the implementation of LLVManim as a 3-layer architecture: **Ingestion**, **Transformation**, and **Presentation**. The Ingestion layer collects LLVM IR and metadata. The transformation layer maps the collected IR constructs and trace events to an intermediate scene graph. The presentation layer renders the scene graph with Manim animations and exposes a small API for interactive playback.

1) User Interface: Command-line interface with automatic compilation:

```
# Visualize C program
llvmanim factorial.c --mode cfg
    --output demo.mp4

# Custom speed and colors
```

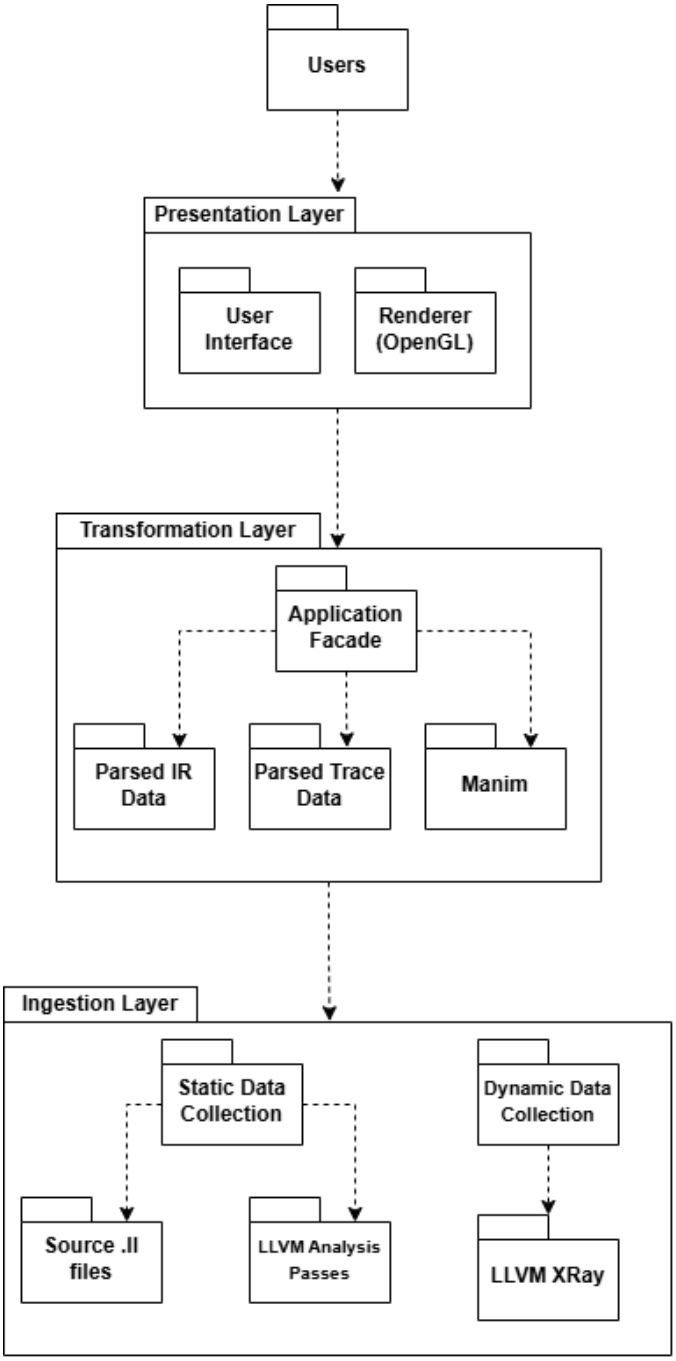

Fig. 3. Model diagram illustrating the three-layer architecture of LLVManim. The Ingestion layer collects LLVM IR and metadata, the Transformation layer maps it to an intermediate scene graph, and the Presentation layer renders it with Manim animations.

```
llvmanim quicksort.c --mode memory
    --speed 0.75 --palette dark
    --output sort.mp4

# Interactive mode
llvmanim recursive.c --mode trace
    --interactive
```

2) Presentation Layer: The Presentation layer renders the scene graph using Manim.

Key operations:

- **Manim Code Generation:** Translate scene graph into Python code defining a Manim `Scene` class. Visual elements become `Mobject` instances with animation commands.
- **Rendering Pipeline:** Invoke Manim to produce MP4 video or enable interactive preview with keyboard controls.
- **Customization API:** Provide configuration options using command line flags and YAML files:
  - `--speed <factor>`: Animation playback speed
  - `--palette <name>`: Color scheme (dark, light, colorblind)
  - `--mode <type>`: Visualization mode (cfg, memory, trace)
  - `--verbose`: Include IR instruction text

3) Transformation Layer: The Transformation layer maps LLVM constructs to an intermediate scene graph for Manim rendering.

Key operations:

- **Scene Graph Construction:** Create hierarchical representation where nodes correspond to visual elements (basic blocks, stack frames, memory cells) with properties for position, appearance, and timing.
- **Layout Algorithm:** Apply hierarchical layout to position CFG nodes while minimizing edge crossings and maintaining logical flow direction. We use a modified Sugiyama framework with constraints for common patterns like if-else branches and loop back-edges.
- **Instruction-to-Animation Mapping:** Define animation primitives for each LLVM instruction:
  - `alloca`: Create stack frame with variable label
  - `load`/`store`: Animate arrow to/from memory location
  - `call`: Push new stack frame with fade-in
  - `ret`: Pop stack frame with fade-out
  - `br`: Highlight control flow edge with color change
- **Timing Coordination:** Calculate timing offsets for animation events (default 1 second per instruction, user-configurable).

4) Ingestion Layer: The Ingestion layer extracts program structure from LLVM IR. We use the `llvmlite` Python binding to parse .ll files generated by clang with the `-g` flag for debug metadata.

Key operations:

- **IR Parsing:** Load LLVM modules and extract functions, basic blocks, and instructions.

- **Control Flow Extraction:** Build CFGs by analyzing branch instructions and identifying successor relationships between blocks.
- **Metadata Collection:** Extract debug information like source line numbers, variable names, and types from LLVM debug intrinsics.
- **Memory Operation Identification:** Catalog memory-related instructions including `load`, `store`, `alloca`, and `call` for tracking memory access patterns and stack operations.
- **Output:** Structured representation with CFG, instruction list, and metadata stored in Python dictionaries mapping basic block IDs to instruction sequences.

The tool compiles .c/.cpp files to LLVM IR automatically or processes .ll files directly. Error messages indicate unsupported features with workarounds.

V. Projected Timeline

Week	Task
W_1 $[2/08 - 2/14]$	• Project proposal draft complete. • Development environment setup.
W_2 $[2/15 - 2/21]$	• LLVM Parsing
W_3 $[2/22 - 2/28]$	• Stack Frame Animation
W_4 $[3/01 - 3/07]$	• Heap Animation
W_5 $[3/08 - 3/14]$	• User Interface
W_6 $[3/15 - 3/21]$	• Interactive feature: Step,Pause
W_7 $[3/22 - 3/28]$	• Animation Speed and Color
W_8 $[3/29 - 4/04]$	• Code Testing
W_9 $[4/04 - 4/10]$	• Code Testing
W_{10} $[4/11 - 4/17]$	• Additional Feature

TABLE I
PROJECTED TIMELINE FOR LLVMANIM DEVELOPMENT.

VI. Feedback

> [1] "The figures are helpful for showing how the existing tools work, but for the proposal make sure you show how you want the visualized code to work. This could be through making a mockup or setting up input for Manim to generate something that looks like what you want. You should also think about the pipeline for what data you are planning on getting from LLVM, how you are going to get it, and how you will feed it into Manim."

This feedback remains unaddressed at present, primarily due to time constraints during the proposal development phase. We intend to create a mockup of the intended output in the near future to better illustrate our vision for the final product.

> [2] "You have figures for some existing tools. Make sure you talk about them in the proposal and what their limitations are."

Acknowledgment

We would like to thank Dr. James Daly for his guidance, feedback, and support throughout the development of this work.

References

[1] P. Guo, "Python Tutor: Visualize Python, Java, C, C++, and JavaScript code execution," https://pythontutor.com/, 2024, accessed: 2024-02-24.

[2] P. J. Guo, "Online python tutor: embeddable web-based program visualization for cs education," in *Proceeding of the 44th ACM Technical Symposium on Computer Science Education*, ser. SIGCSE '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 579–584. [Online]. Available: https://doi.org/10.1145/2445196.2445368

[3] ——, "Codeopticon: Real-time, one-to-many human tutoring for computer programming," in *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*, ser. UIST '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 599–608. [Online]. Available: https://doi.org/10.1145/2807442.2807469

[4] C. Lattner and V. Adve, "LLVM: A compilation strategy for lifelong program analysis and transformation," in *Proceedings of the 2nd IEEE Annual CGO and ACM SIGPLAN Annual CC Symposium*, ser. CGO-CC '04. Palo Alto, CA, USA: IEEE Computer Society, 2004, pp. 60–72.

[5] LLVM Project, *LLVM Language Reference Manual*, LLVM Project, 2026, accessed: Feb. 11, 2026. [Online]. Available: https://llvm.org/docs/LangRef.html

[6] J. Ellson, E. Gansner, E. Koutsofios, S. North, and G. Woodhull, "Graphviz - Open Graph Visualization Software," https://graphviz.org/, 2001, accessed: [Insert Date Here].

[7] G. Sanderson, "manim," 2015, accessed 2026-02-24. [Online]. Available: https://github.com/3b1b/manim

[8] ——, "3blue1brown," Website, 2026, accessed: Feb. 13, 2026. [Online]. Available: https://www.3blue1brown.com/

[9] ——, "3blue1brown," Youtube Channel, March 2015, accessed: Feb. 13, 2026. [Online]. Available: https://www.youtube.com/@3blue1brown